

基于 Docker 的 Hadoop 分布式文件系统（HDFS）搭建与基本操作

1 背景介绍

随着大数据技术的快速发展，海量数据的存储与处理成为计算机领域的重要研究方向。传统的单机存储方式已经难以满足大规模数据处理的需求，分布式存储与计算技术逐渐成为主流解决方案。其中，Apache Hadoop 是当前应用最广泛的大数据处理框架之一。

Hadoop 主要由分布式文件系统（Hadoop Distributed File System, HDFS）和分布式计算框架（MapReduce）组成。其中，HDFS 负责对海量数据进行可靠、高效的分布式存储，通过数据块分割与多副本机制，实现数据的高容错性和高可用性。在 HDFS 中，系统采用主从架构，由 NameNode 负责元数据管理，由多个 DataNode 负责实际数据存储。

在实际教学与实验环境中，由于物理服务器资源有限，通常采用虚拟化或容器化技术来模拟多节点分布式系统。Docker 作为一种轻量级容器技术，能够快速构建多个隔离的运行环境，非常适合用于搭建 Hadoop 分布式集群。本实验基于 Docker 容器模拟多台虚拟主机，通过构建 Hadoop 集群，实现 HDFS 的基本功能操作。

本实验的主要内容包括：搭建 Hadoop 分布式环境、完成 HDFS 的基本文件操作（如创建目录、上传文件、查看文件内容与属性、删除文件等），并通过编程方式实现对 HDFS 文件的读取功能。

2 实验过程

2.1 搭建 Hadoop 集群

本实验采用 Docker 容器技术模拟多节点分布式环境，在本地计算机上构建三节点 Hadoop 集群，包括一个主节点（NameNode）和两个从节点（DataNode）。具体搭建过程如下：

首先，利用 Docker Compose 工具创建三个容器，分别命名为 `hadoop-master`、`hadoop-slave1` 和 `hadoop-slave2`，并通过自定义网络实现容器之间的互联互通。三个容器位于同一虚拟网段中，从而能够通过主机名进行相互访问，模拟真实分布式集群环境。

`docker-compose.yml` 在附录中，运行

```
docker compose up -d
```

```

● (base) PS C:\Users\PC\hadoop-cluster> docker compose up -d
[+] Running 4/4
 ✓ Network hadoop-cluster_hadoop-net Created
 ✓ Container hadoop-slave1 Started
 ✓ Container hadoop-slave2 Started
 ✓ Container hadoop-master Started
    
```

图 1 Docker Compose 工具命令行截图

●	hadoop-cluster	-	-	-
●	hadoop-slave2	5a314ee6e7ef	ubuntu:22.04	
●	hadoop-master	c6cef0ab31ee	ubuntu:22.04	19000:9000 ↗ 18070:9870 ↗ Show less
●	hadoop-slave1	cbd3dd660d6f	ubuntu:22.04	

图 2 Docker Desktop 运行界面截图

随后，在每个容器中安装必要的软件环境，包括 Java 运行环境（OpenJDK）、SSH 服务以及相关工具（rsync、vim 等）。其中，Java 环境是 Hadoop 运行的基础，SSH 服务用于实现节点之间的远程通信与任务调度。

apt update

apt install -y openjdk-11-jdk openssh-server rsync vim wget

mkdir -p /var/run/sshhd

service ssh start

下一步，在主节点（hadoop-master）中生成 SSH 密钥对，并将公钥分发至所有节点，实现主节点对从节点的免密登录。该步骤是 Hadoop 集群正常运行的重要前提，因为 Hadoop 在启动和运行过程中需要通过 SSH 在各节点之间执行远程命令。

在 master 生成密钥：ssh-keygen -t rsa

```

# ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Created directory '/root/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:T+kHNT0czsws0IYLphoSnw50aXBk0jhMzMNxGAj1DfI root@hadoop-master
The key's randomart image is:
+---[RSA 3072]-----+
|
|
|
|
|
|
|
|
|
|
+---[SHA256]-----+
    
```

图 3 在 master 生成密钥截图

复制公钥: `cat ~/.ssh/id_rsa.pub`

```
ssh-rsa xxx
```

分发到三个节点, 在 master 自己:

```
mkdir -p ~/.ssh
echo "刚才复制的整行公钥" >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

在 slave1 和在 slave2 加上 `service ssh restart`。

回到 master 测试免密:

```
ssh hadoop-slave1 hostname
ssh hadoop-slave2 hostname
```

```
# ssh hadoop-slave1 hostname
The authenticity of host 'hadoop-slave1 (172.20.0.11)' can't be established.
ED25519 key fingerprint is SHA256:
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hadoop-slave1' (ED25519) to the list of known hosts.
hadoop-slave1
# ssh hadoop-slave2 hostname
The authenticity of host 'hadoop-slave2 (172.20.0.12)' can't be established.
ED25519 key fingerprint is SHA256:
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hadoop-slave2' (ED25519) to the list of known hosts.
hadoop-slave2
```

图 5 回到 master 测试免密结果截图

随后, 在主节点中下载并安装 Hadoop 软件, 并将其解压至指定目录(`/usr/local/hadoop`)。配置 Hadoop 相关环境变量, 包括 `HADOOP_HOME`、`JAVA_HOME` 和 `PATH` 等, 使系统能够正确调用 Hadoop 命令。

在 master 下载并安装 Hadoop:

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
mv /root/hadoop-3.3.6 /usr/local/Hadoop
```

配置 Hadoop 环境变量：

```
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

在 Hadoop 配置文件中，对集群运行参数进行设置。主要包括：

```
cd $HADOOP_HOME/etc/hadoop
```

- 在 core-site.xml 中配置默认文件系统地址（NameNode 地址）；

```
vim core-site.xml
```

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://hadoop-master:9000</value>
  </property>
</configuration>
```

- 在 hdfs-site.xml 中配置数据副本数及存储路径；

```
vim hdfs-site.xml
```

```
<configuration>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
</configuration>
```

- 在 workers 文件中指定从节点列表；

```
vim workers
```

```
hadoop-slave1
hadoop-slave2
```

- 在 hadoop-env.sh 中配置 Java 环境变量。

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

完成配置后，将 Hadoop 安装目录从主节点分发至各从节点，保证所有节点环境一致。

在 3 个容器里执行：

```
mkdir -p /usr/local/hadoop/tmp/dfs/name
mkdir -p /usr/local/hadoop/tmp/dfs/data
```

在 master 里执行：

```
scp -r /usr/local/hadoop hadoop-slave1:/usr/local/
scp -r /usr/local/hadoop hadoop-slave2:/usr/local/
```

随后，在主节点执行 NameNode 格式化操作，并启动 HDFS 服务。

```
/usr/local/hadoop/sbin/start-dfs.sh
```

最后，通过执行 `jps` 命令检查各节点进程运行情况，在主节点可以看到 NameNode 和 SecondaryNameNode，在从节点可以看到 DataNode，说明 Hadoop 集群搭建成功。

```
C:\Windows\System32>docker exec -it hadoop-master bash -c "jps"
3441 Jps
3219 SecondaryNameNode
2935 NameNode

C:\Windows\System32>docker exec -it hadoop-slave2 bash -c "jps"
311 DataNode
462 Jps

C:\Windows\System32>docker exec -it hadoop-slave1 bash -c "jps"
480 DataNode
631 Jps
```

图 6 检查各节点进程运行情况截图

访问 <http://localhost:18070/> 可以看到可视化界面。

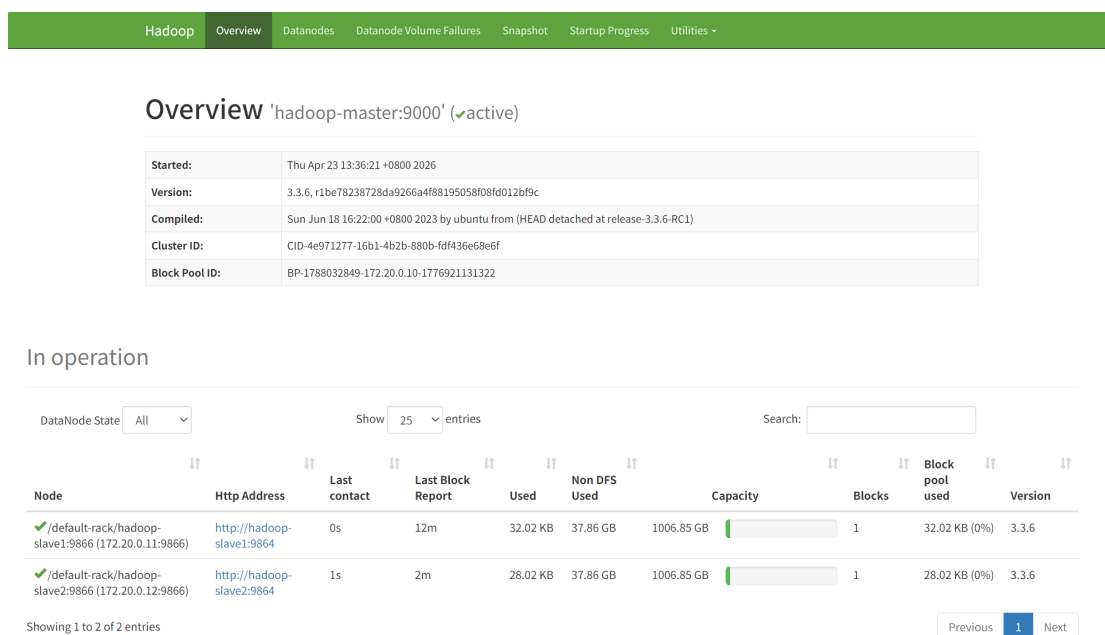


图 7 Hadoop 网页监控截图

2.2 HDFS 基本操作过程

在成功搭建 Hadoop 集群并启动 HDFS 服务后，进一步对分布式文件系统进行基本操作实验，包括目录创建、文件上传、文件查看以及文件删除等操作。

首先，在 HDFS 中创建目录/Mydoc:

```
hdfs dfs -mkdir /Mydoc
```

随后，在本地创建测试文件 MyFile，内容为“hello hadoop”:

```
echo "hello hadoop" > MyFile
```

在未将文件上传至 HDFS 前，尝试直接查看文件内容:

```
hdfs dfs -cat /Mydoc/MyFile
```

系统返回: No such file or directory，说明该文件尚未存在于 HDFS 中。

接着，将本地文件上传到 HDFS 的/Mydoc 目录中:

```
hdfs dfs -put MyFile /Mydoc
```

上传完成后，通过列出目录内容验证文件是否存在:

```
hdfs dfs -ls /Mydoc
```

系统显示: /Mydoc/MyFile，说明文件上传成功。

随后，查看文件的详细信息，包括文件名、大小、副本数以及创建时间:

```
hdfs dfs -stat "%n %b %o %r %y" /Mydoc/MyFile
```

输出结果表明文件大小为 13 字节，副本数为 2，说明 HDFS 已按照配置进行数据复制。

接下来，读取文件内容并输出到终端:

```
hdfs dfs -cat /Mydoc/MyFile
```

终端显示: hello hadoop，说明文件内容存储正确。

最后，对文件及目录进行删除操作:

```
hdfs dfs -rm /Mydoc/MyFile
```

```
hdfs dfs -rmdir /Mydoc
```

执行完成后，目录/Mydoc 被成功删除，说明 HDFS 文件管理功能正常。

```
# hdfs dfs -mkdir /Mydoc
# echo "hello hadoop" > MyFile
# hdfs dfs -cat /Mydoc/MyFile
cat: `/Mydoc/MyFile': No such file or directory
# hdfs dfs -put MyFile /Mydoc
# hdfs dfs -ls /Mydoc
Found 1 items
-rw-r--r--    2 root supergroup          13 2026-04-23 05:47 /Mydoc/MyFile
# hdfs dfs -stat "%n %b %o %r %y" /Mydoc/MyFile
MyFile 13 134217728 2 2026-04-23 05:47:13
# hdfs dfs -cat /Mydoc/MyFile
hello hadoop
# hdfs dfs -rm /Mydoc/MyFile
Deleted /Mydoc/MyFile
# hdfs dfs -rmdir /Mydoc
```

图 8 对分布式文件系统进行基本操作截图

2.3 基于 Java 的 HDFS 文件读取实现

在完成 Hadoop 集群搭建以及 HDFS 基本操作之后，进一步使用 Java 程序实现对 HDFS 文件内容的读取，以验证 Hadoop API 的使用方法 & 分布式文件访问机制。

本实验中，编写 MyFSDataInput 类，通过 org.apache.hadoop.fs.FileSystem 和 FSDataInputStream 打开 HDFS 中的文件，并使用 readLine()方法逐行读取文件内容。程序首先根据 Hadoop 配置获取文件系统对象，然后打开 HDFS 中的目标文件/Mydoc/MyFile，最后将读取到的内容输出到终端。MyFSDataInput.java 在附录中。

程序编写完成后，使用 Hadoop 提供的类路径进行编译：

```
javac -classpath `hadoop classpath` MyFSDataInput.java
```

编译成功后，运行程序：

```
java -classpath .:`hadoop classpath` MyFSDataInput
```

运行结果如下：

```
# javac -classpath `hadoop classpath` MyFSDataInput.java
# java -classpath .:`hadoop classpath` MyFSDataInput
2026-04-23 05:56:14,333 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
hello hadoop
```

图 9 java 正确访问 HDFS 中的文件截图

实验结果表明，程序能够正确访问 HDFS 中的文件，并成功读取文件内容，说明 HadoopJava 开发环境配置正确，HDFS 文件读取功能实现成功。

通过该部分实验，我进一步掌握了 Hadoop Java API 的基本使用方法，加深了对 HDFS 文件读取机制的理解。

3 实验结果与分析

通过本次实验，我成功在本地基于 Docker 容器搭建了一个三节点的 Hadoop 分布式集群，并实现了 HDFS 的基本功能操作以及基于 Java 的文件读取功能。

在集群搭建过程中，通过 Docker Compose 构建了一个包含一个主节点（NameNode）和两个从节点（DataNode）的分布式环境，并通过 SSH 实现节点之间的免密通信。通过执行 jps 命令，可以在主节点观察到 NameNode 和 SecondaryNameNode 进程，在从节点观察到 DataNode 进程，说明集群运行正常。

在 HDFS 基本操作实验中，我成功完成了目录创建、文件上传、文件内容读取、文件信息查询以及文件删除等操作。实验结果表明：

- HDFS 能够正确创建和管理分布式目录结构；
- 文件上传后可以在多个节点间进行副本存储，提高数据可靠性；
- 通过 hdfs dfs -stat 命令可以查看文件副本数，验证数据复制机制；
- 文件内容读取与删除操作均能够正常执行，说明文件系统运行稳定。

在 Java 编程部分，通过编写 MyFSDataInput 程序，实现了对 HDFS 文件的读取功能。

程序能够成功访问分布式文件系统，并将文件内容输出到终端，验证了 Hadoop Java API 的正确使用。

总体来看，本实验成功验证了 Hadoop 分布式文件系统的基本功能以及 Java 接口调用的可行性。

4 实验总结

本次实验基于 Docker 容器成功搭建了三节点 Hadoop 分布式集群，并完成了 HDFS 的基本操作与 Java 编程实践。通过实验，我掌握了 Hadoop 集群的部署流程、HDFS 文件管理命令以及分布式存储的基本原理，同时学会了利用 Hadoop API 实现对 HDFS 文件的访问。实验过程中对分布式系统的运行机制、数据副本策略及节点协同工作方式有了更直观的认识，也提升了在实际环境中配置与排查问题的能力，为后续学习大数据处理技术奠定了基础。